

Cloud Vendor Selection — Technical Analysis

Project: Commute Tracker v0.1 · **Lens:** Greenfield, technical merit only · **Date:** 2026-05-27

Framing

This revision discards the prior brief's leaning on prior planning artifacts and toolchain familiarity. The app has been planned but not implemented — so vendor selection is a clean greenfield choice. The only inputs that count here are **technical fit for the workload** and **implementation effort from scratch**.

App profile — Flutter Android app, iOS later. Offline-first: Drift (local SQLite) is the source of truth. Cloud is a thin backup behind 3 REST endpoints: POST /trips/sync , DELETE /trips/{id} , GET /trips/restore . Single user, hobby scale.

Hard constraints — single cloud vendor; auth should be simple.

Technical Trade-off Matrix

Dimension	All-AWS	All-Firebase	Edge
Auth UX	Email + 6-digit code (Cognito passwordless)	Google one-tap on Android, native UI on iOS	Firebase · major
Auth implementation effort	Custom auth flow: 3 Lambda triggers (Define / Create / Verify) + SES sender + Cognito User Pool config. Or paid Managed Passwordless tier.	flutterfire configure , drop in FirebaseAuthProvider , done. Hours of work.	Firebase · major
Flutter SDK	Community package amazon_cognito_identity_dart_2 . Maintained, but not first-party; trails Cognito features.	FlutterFire — first-party (Google owns Flutter). Tracks SDK changes immediately.	Firebase
Backend compute	Lambda + API Gateway via SAM. Cold starts ~200-400ms on Node 24. Mature, predictable.	Cloud Functions (2nd gen) HTTP triggers. Similar cold-start profile. Less ceremony to deploy.	Firebase
Database fit	DynamoDB. Requires single-table design thinking even for 3 access patterns; PK/SK and GSI planning up-front.	Firestore. Document model maps directly to a trip object. Queries are obvious. No schema design phase.	Firebase

Dimension	All-AWS	All-Firebase	Edge
laC / deploy tooling	SAM templates, well-documented, declarative. Powerful when you grow.	Firebase CLI: <code>firebase deploy</code> . Less powerful, but matches the scope here.	Tie
Local development	SAM local invoke + DynamoDB Local. Workable but rough; emulating IAM is painful.	Firebase Emulator Suite (Auth + Firestore + Functions + Storage). Hot-reload, UI included. Notably better DX.	Firebase
Security model	IAM policies + Cognito-authorized API Gateway. Granular but verbose; lots of JSON.	Firebase Security Rules — declarative, written next to the schema. Simpler for this scope.	Firebase
Observability	CloudWatch logs + metrics + X-Ray traces. Industrial-strength; overkill here but free.	Cloud Logging + Firebase Console. Lighter, sufficient at this scope.	Tie
Backend type safety	TypeScript Lambda with <code>@aws-sdk/lib-dynamodb</code> + <code>zod</code> . Strong types end-to-end.	TypeScript Cloud Functions + Firestore admin SDK + <code>zod</code> . Equally strong.	Tie
Cost at hobby scale	~\$0 — Lambda + DynamoDB + Cognito + API Gateway free tier all cover single-user use.	~\$0 — Spark plan covers Auth + Firestore + Functions for hobby use.	Tie
Cross-platform parity (iOS later)	Email OTP works identically on both. No deep-link redirects.	FlutterFire on iOS is equally first-party. Apple Sign-In available as parallel option.	Tie
Vendor portability	App talks REST → backend is swappable.	Cloud Functions HTTPS triggers → also REST → also swappable. Risk: Firestore SDK is tempting to use direct from client (breaks portability).	AWS (slight)
Learning curve from zero	Steeper: IAM, SAM, single-table DynamoDB, Cognito triggers, SES.	Gentler: one CLI, one console, simpler primitives.	Firebase
Backend control / extensibility	Anything AWS offers (SQS, EventBridge, Step Functions) plugs in directly.	Anything GCP offers (Pub/Sub, Cloud Tasks) plugs in, but smaller ecosystem.	AWS

Decision Drivers (ties stripped)

Firestore wins (7, two major)

- Auth UX (**major**)
- Auth build effort (**major**)
- First-party Flutter SDK
- Database fit for simple writes/reads
- Local development experience
- Security model simplicity
- Lower learning curve

AWS wins (2, none major)

- Backend extensibility / ecosystem depth
- Slight edge on vendor portability (less temptation to leak SDK into client)

AWS's wins are real but only matter if the app grows into territory that **doesn't exist in the v0.1 scope**: complex event processing, multi-service workflows, queue-based ingestion. For 3 REST endpoints behind a thin offline-first app, that capacity sits unused.

For a personal Flutter app with 3 REST endpoints and offline-first data, which vendor matches the workload?

✓ Recommendation — All-Firebase

1. **Workload-vendor fit.** Firestore's primitives are sized for this app. AWS's primitives are sized for systems an order of magnitude larger; using them here means absorbing complexity that produces no value.
2. **Auth is the load-bearing dimension.** The original constraint was "auth must be simple". Google one-tap via FlutterFire is genuinely the simplest mobile auth available today. Cognito email-OTP is simple *as far as Cognito goes*, but still requires Lambda triggers, SES wiring, and a community Flutter SDK.
3. **Time-to-first-deploy.** Firestore: hours. AWS: days. For a personal project where the goal is to ship and use the app, that delta is large.
4. **SDK first-party-ness.** FlutterFire is built by the team that builds Flutter. The Cognito Dart SDK is community-maintained and has historically lagged Cognito feature releases. On a long-lived personal app, this matters.

When AWS is still the right call

The recommendation flips to AWS if any of these are true:

- **Learning goal** — you want hands-on experience with Lambda, DynamoDB, IAM, or SAM for career or curiosity reasons. The project becomes a learning vehicle, and AWS pays off.
- **Growth plan** — you intend to expand into queue processing, ML pipelines, multi-service event flows, or anything that benefits from AWS's broader service catalog.
- **Vendor stance on Google** — if you'd rather not be on Google's stack for non-technical reasons (privacy posture, vendor diversity, etc.), that's a legitimate input.

None of these are technical arguments for the workload as currently scoped — they're personal/strategic inputs.

Implementation Sketch (All-Firebase)

Rough scope of what Phases 9–11 become under Firebase, for reference:

- **Phase 9 (Auth):** `flutterfire configure`, add `firebase_auth` + `google_sign_in`, wire `signInWithProvider(GoogleAuthProvider)`, store the ID token via `flutter_secure_storage`. Estimated effort: half-day, no spike needed.
- **Phase 10 (Backend):** Three HTTPS Cloud Functions in TypeScript, each validating input with `zod` and writing to Firestore. Auth is automatic via the Firebase Admin SDK's request context. Estimated effort: 1 day.
- **Phase 11 (Sync):** Drift → http client → Cloud Functions endpoints. Same shape as the AWS plan, different base URL. Sync queue logic unchanged. Estimated effort: 1–2 days.

⚠ Pitfalls to Plan Around

- **Don't let Firestore leak into the client.** Use the REST shape (Cloud Functions HTTPS triggers) from the Flutter app — not the `cloud_firestore` SDK. This preserves the offline-first design (Drift is SOT) and keeps backend swappable.
- **Security Rules still need writing.** Even with REST endpoints, lock Firestore down so only the Admin SDK can read/write trip data. Default rules deny — keep them that way.
- **Pin SDK versions.** FlutterFire moves fast; pin to a known-good combination in `pubspec.yaml` to avoid breakage on `flutter pub upgrade`.

Bottom Line

On pure technical merit, with no implementation sunk cost and no toolchain bias, **Firebase fits this workload better than AWS**. The match isn't razor-thin — Firebase wins 7 of the discriminating dimensions, two of them clearly. AWS wins on ecosystem depth and extensibility, neither of which the v0.1 scope exercises.

The decision genuinely flips only if you want this project to be a learning vehicle for AWS, or you have non-technical reasons to avoid Google. Otherwise, the simpler, faster, better-integrated path is Firebase.